Platform Used:

- Google Colab
- Apache Spark (PySpark)

Work Done:

- Loaded the dataset into Spark DataFrame
- Removed duplicate records
- Handled null values
- Applied filters on age, subscription, and region
- Performed aggregation functions (count, sum, avg, min, max)
- Used groupBy operations
- Renamed and modified columns
- Studied DataFrame immutability and shuffle operations
- cleaning and revenue analysis pipeline Built a complete data

#Imported the required PySpark libraries for creating Spark sessions, performing data transformations, and handling timestamp data types.

 # Create Spark Session

#load dataset

```
[19] ✓ 0s
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import TimestampType

[20] ✓ 0s
spark = SparkSession.builder \
    .appName("Week5_Spark_Assignment") \
    .getOrCreate()

[22] ✓ 0s
df = spark.read.csv(
    "/content/sample_data/week5_spark_dataset.csv",
    header=True,
    inferSchema=True
)
```

Display Schema

```
[23] ✓ 0s
df.printSchema()

root
 |-- user_id: integer (nullable = true)
 |-- transaction_date: date (nullable = true)
 |-- username: string (nullable = true)
 |-- email: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- subscription: string (nullable = true)
 |-- city: string (nullable = true)
 |-- region: string (nullable = true)
 |-- store_id: string (nullable = true)
 |-- product_category: string (nullable = true)
 |-- status: string (nullable = true)
 |-- price: integer (nullable = true)
 |-- sale_amount: integer (nullable = true)
```

#Display Data

```
[24] ✓ 0s
df.show()
```

user_id	transaction_date	username	email	age	subscription	city	region	store_id	product_category	status	price	sale_amount
1	2025-01-01	john	john@gmail.com	25	Premium	Delhi	West	S101	Electronics	Active	500	500
2	2025-01-02	alice	alice@gmail.com	32	Basic	Mumbai	South	S102	Furniture	Active	700	700
3	2025-01-03	bob	NULL	28	Premium	Delhi	West	S101	Electronics	NULL	300	300
4	2025-01-04	NULL	emma@gmail.com	22	Premium	Chennai	East	S103	Clothing	Active	NULL	450
5	2025-01-05	mike	mike@gmail.com	40	Basic	Kolkata	North	S104	Electronics	Inactive	900	900
1	2025-01-01	john	john@gmail.com	25	Premium	Delhi	West	S101	Electronics	Active	500	500
6	2025-01-06	sarah	sarah@gmail.com	29	Premium	Mumbai	South	S102	Furniture	Active	NULL	800
7	2025-01-07	david	david@gmail.com	35	Premium	Delhi	West	S101	Electronics	Active	600	600
8	2025-01-08	chris	chris@gmail.com	18	Premium	Chennai	East	S103	Clothing	Active	350	350
9	2025-01-09	eva	eva@gmail.com	30	Basic	Kolkata	North	S104	Furniture	Inactive	400	400

DATA CLEANING

Remove Duplicates based on user_id and transaction_date & # Fill null values& # Remove rows with null email or empty username

```
[26] ✓ 0s
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
```

+ Code + Text

```
[28] ✓ 0s
df_clean = df_clean.na.fill({
    "status": "Unknown",
    "price": 0
})
```

```
[29] ✓ 0s
df_clean = df_clean.filter(
    col("email").isNotNull() &
    col("username").isNotNull() &
    (col("username") != "")
)
```

FILTERING OPERATIONS

Age between 18 and 30 with Premium subscription

```
[30]
✓ 0s
premium_users = df_clean.filter(
    (col("age").between(18, 30)) &
    (col("subscription") == "Premium")
)

premium_users.show()
```

```
+-------+----------------+--------+--------------+---+------------+-------+------+--------+----------------+------+-----+-----------+
|user_id|transaction_date|username|         email|age|subscription|   city|region|store_id|product_category|status|price|sale_amount|
+-------+----------------+--------+--------------+---+------------+-------+------+--------+----------------+------+-----+-----------+
|      1|      2025-01-01|    john| john@gmail.com| 25|     Premium|  Delhi|  West|    S101|     Electronics|Active|  500|        500|
|      6|      2025-01-06|   sarah|sarah@gmail.com| 29|     Premium| Mumbai| South|    S102|       Furniture|Active|    0|        800|
|      8|      2025-01-08|   chris|chris@gmail.com| 18|     Premium|Chennai|  East|    S103|        Clothing|Active|  350|        350|
+-------+----------------+--------+--------------+---+------------+-------+------+--------+----------------+------+-----+-----------+
```

#Region = West

```
[31]
✓ 0s
west_sales = df_clean.filter(col("region") == "West")

west_sales.show()
```

```
+-------+----------------+--------+--------------+---+------------+-----+------+--------+----------------+------+-----+-----------+
|user_id|transaction_date|username|         email|age|subscription| city|region|store_id|product_category|status|price|sale_amount|
+-------+----------------+--------+--------------+---+------------+-----+------+--------+----------------+------+-----+-----------+
|      1|      2025-01-01|    john| john@gmail.com| 25|     Premium|Delhi|  West|    S101|     Electronics|Active|  500|        500|
|      7|      2025-01-07|   david|david@gmail.com| 35|     Premium|Delhi|  West|    S101|     Electronics|Active|  600|        600|
+-------+----------------+--------+--------------+---+------------+-----+------+--------+----------------+------+-----+-----------+
```

AGGREGATION FUNCTIONS

Count Records

```
[32]
✓ 0s
df_clean.select(count("*").alias("total_records")).show()
```

```
+-------------+
|total_records|
+-------------+
|            7|
+-------------+
```

Sum Sale Amount

```
[33]
✓ 0s
df_clean.select(sum("sale_amount").alias("total_sales")).show()
```

```
+-----------+
|total_sales|
+-----------+
|       4250|
+-----------+
```

Average Sale Amount

```
[34] ✓ 0s
df_clean.select(avg("sale_amount").alias("avg_sales")).show()
```

```
+------------------+
|        avg_sales|
+------------------+
|607.1428571428571|
+------------------+
```

Minimum Price

```
[35] ✓ 0s
df_clean.select(min("price").alias("min_price")).show()
```

```
+---------+
|min_price|
+---------+
|        0|
+---------+
```

Maximum Price

```
[36] ✓ 0s
df_clean.select(max("price").alias("max_price")).show()
```

```
+---------+
|max_price|
+---------+
|      900|
+---------+
```

GROUP BY OPERATIONS

Average sale_amount by product_category in West region

```
[37] ✓ 0s
west_sales.groupBy("product_category") \
    .agg(avg("sale_amount").alias("avg_sale_amount")) \
    .show()
```

```
+----------------+---------------+
|product_category|avg_sale_amount|
+----------------+---------------+
|     Electronics|          550.0|
+----------------+---------------+
```

Count records by city

```
[39]
✓ 0s
city_count = df_clean.groupBy("city") \
    .agg(count("*").alias("city_count"))

city_count.show()
```

```
+-------+----------+
|   city|city_count|
+-------+----------+
|Chennai|         1|
| Mumbai|         2|
|Kolkata|         2|
|  Delhi|         2|
+-------+----------+
```

Cities having count > 100

```
[40]
✓ 0s
city_count.filter(col("city_count") > 100).show()
```

```
+----+----------+
|city|city_count|
+----+----------+
+----+----------+
```

Total Revenue by Store

```
[41]
✓ 0s
df_clean.groupBy("store_id") \
    .agg(sum("price").alias("total_revenue")) \
    .show()
```

```
+--------+-------------+
|store_id|total_revenue|
+--------+-------------+
|    S102|          700|
|    S104|         1300|
|    S101|         1100|
|    S103|          350|
+--------+-------------+
```

MULTIPLE AGGREGATIONS

```
[42]
✓ 0s
df_clean.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    avg("price").alias("avg_price")
).show()
```

```
+---------+---------+------------------+
|min_price|max_price|         avg_price|
+---------+---------+------------------+
|        0|      900|492.85714285714283|
+---------+---------+------------------+
```

SCHEMA MODIFICATION

Cast timestamp column (if exists)

Example for raw_timestamp column

```
# df_clean = df_clean.withColumn(
# "event_time",
# col("raw_timestamp").cast(TimestampType())
# ).drop("raw_timestamp")
```

 # Rename Column Example

```
[43]
✓ 0s
df_clean = df_clean.withColumnRenamed(
    "transaction_date",
    "txn_date"
)
```

FINAL DATA PROCESSING PIPELINE

```
[44]
✓ 0s
final_result = (
    df.dropDuplicates(["user_id", "transaction_date"])
      .na.fill({"price": 0})
      .groupBy("store_id")
      .agg(sum("price").alias("total_revenue"))
)

final_result.show()

# Stop Spark Session
spark.stop()
```

```
+--------+-------------+
|store_id|total_revenue|
+--------+-------------+
|    S102|          700|
|    S104|         1300|
|    S101|         1400|
|    S103|          350|
+--------+-------------+
```